



Research Diary

Can neural networks reliably estimate kerning, and how much does context matter?

Lab: Computational Intelligence Lab (CILab)

Supervisor: Dr. Gennaro Vessio

Project: Learning to Kern: Comparison of Architectures for Optimal Letter Space

Collaborators:

- Angelantonio Fedele Murolo – M.Sc. student
- Emanuele Colonna – Ph.D. student

Start date: Wednesday 10th June, 2026

End date: Wednesday 2th September, 2026

Friday 4th September, 2026

Contents

1	Why Kerning, and What We Set Out to Do	2
2	Data Acquisition: Choosing a Font-First Pipeline	2
3	Rasterization and the Shape of the Ground Truth	3
4	The First Pivot: From Sparse Kern Pairs to the Full N^2 Target	4
5	ResNet18 Pre-training and Feature Extraction	5
6	Pairwise Model: The Strongest Single Architecture	5
7	Set-wise Model: The Expected Advantage That Did Not Materialize	6
8	Ensemble and Hybrid Models: Searching for Global Signal	7
9	Results	9
10	Explainability: What the Models Actually Learned	17
11	Interactive Demo	20
12	Conclusions	22
13	Discussion	22

Why Kerning, and What We Set Out to Do

The starting point of this project is *Learning to Kern: Set-wise Estimation of Optimal Letter Space* by Nakatsuru and Uchida (2024). The paper frames kerning as a machine-learning problem: given a proportional Latin font, predict the horizontal distance between the centers of adjacent glyphs for every ordered letter pair. For the 52 letters used in the paper (A – Z and a – z), this means predicting $52 \times 52 = 2704$ distances per font. The correct distance depends on the specific pair, on the font style, and on the surrounding typographic context; there is no closed-form rule, which is why type designers still kern manually or with imperfect heuristics such as FontForge’s optical spacing.

The paper proposes two models. The *pairwise* model concatenates two frozen ResNet18 features into a 1024-dimensional vector, adds the letter identities as one-hot conditions, and regresses a single spacing value through a three-layer DNN. The *set-wise* model instead builds all N^2 pair tokens from a font, projects them to 32 dimensions, and processes them jointly with a Transformer so that self-attention can enforce global consistency. On 2558 Google Fonts the paper reports a pairwise MAE of 13.27 pixels and a set-wise MAE of 5.32 pixels, with the set-wise model clearly dominant.

Our goal was to replicate that framework on a substantially larger corpus, and then to probe the central claim—that global, set-wise reasoning is what makes kerning accurate. In our setting that probe ended up reversing the conclusion of the original paper, and the reasons are worth documenting in detail.

Data Acquisition: Choosing a Font-First Pipeline

The first decision concerned where the ground truth would come from. An initial suggestion was to start from a Kaggle dataset of rendered text, but that data is organized around OCR training, with whole strings and labels aimed at recognition rather than the per-pair geometric measurements that kerning requires. Building a kerning dataset from it would have meant reverse-engineering a spacing signal it was never designed to preserve. We therefore adopted a *font-first* strategy.

The corpus is Google Fonts, retrieved through the official WebFonts API. After querying the full catalog and removing monospaced families (where kerning is trivial by construction), we downloaded the available TTF variants for the remaining proportional families. A metadata CSV kept track of family, stem, category, size, and local path, which also made the acquisition stage easy to audit. This phase produced:

- 1868 proportional font families and 7284 downloaded TTF variants, with zero download failures;
- a catalog CSV with metadata, download status, and paths;
- a kerning-pair CSV extracted from GPOS PairPos tables, with the legacy `kern` table used as a fallback.

Kerning extraction was restricted to the same 52 Latin characters as the paper, and only pairs with a nonzero adjustment were stored explicitly. This yielded 715,826 rows, corresponding to a kern rate of 2.7% over all declared Latin-52 pairs. The distinction between *explicitly kerned*

pairs and *all* pairs is central to the rest of the project: storing only the former is natural for bookkeeping, but it is not the same thing as the full supervision signal a model needs.

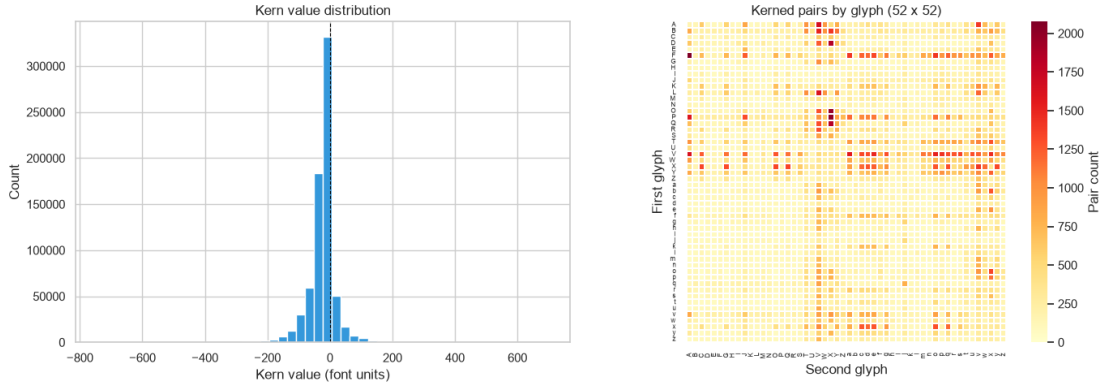


Figure 1: Distribution of the extracted kerning values in font units (left), restricted to the nonzero GPOS/kern entries, and the 52×52 count matrix of explicitly kerned letter pairs (right). The distribution is sharply peaked near zero and slightly skewed negative; the heatmap shows that kerning concentrates on a minority of glyphs—notably T, V, W, and Y—which act as frequent first glyphs in adjusted pairs.

Rasterization and the Shape of the Ground Truth

Every font was rasterized so that each of the 52 glyphs became a 256×256 binary image with a shared baseline at $y = 160$. A per-font scale was chosen so that all glyphs fit inside the canvas while preserving aspect ratio and vertical alignment. Fonts for which no valid shared scale existed, or that failed during FreeType rendering, were dropped rather than rendered inconsistently.

For each font we stored the glyph images, the character identities, the canvas centroids, the centroids relative to the pen position, the advance widths, and the pair-level distances. The ground-truth spacing for an ordered pair (L, R) is

$$t(L, R) = \text{advance}(L) + k(L, R) \cdot s + \text{centroid}_x(R) - \text{centroid}_x(L), \quad (1)$$

where $k(L, R)$ is the explicit kerning adjustment in font units, s is the per-font scale, and $k(L, R) = 0$ when no explicit entry exists. The key phrase is “ $k = 0$ when absent”: a pair without a GPOS entry is not a pair without a target, it is a pair whose target is determined by the advances and centroids alone.

That distinction is exactly what the first version of the pipeline missed. The early implementation built targets only for pairs with a nonzero kerning entry—roughly 99 pairs per font out of 2704, discarding about 96% of the data before training even started. It also drifted from the reference paper, whose experiments target all N^2 pairs.

After rasterization, 7263 fonts were accepted and 21 rejected. All accepted fonts have the full set of 52 glyphs. The dataset contains 377,676 glyph images and 19,639,152 pair-level targets.

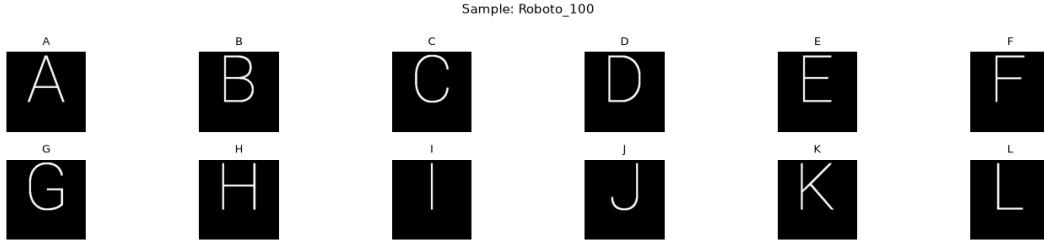


Figure 2: Rasterized glyphs for a representative font (Roboto Thin). The first twelve glyphs (A–L) are shown on a 256×256 canvas with a shared baseline, illustrating the consistency of scale and vertical alignment enforced across the corpus.

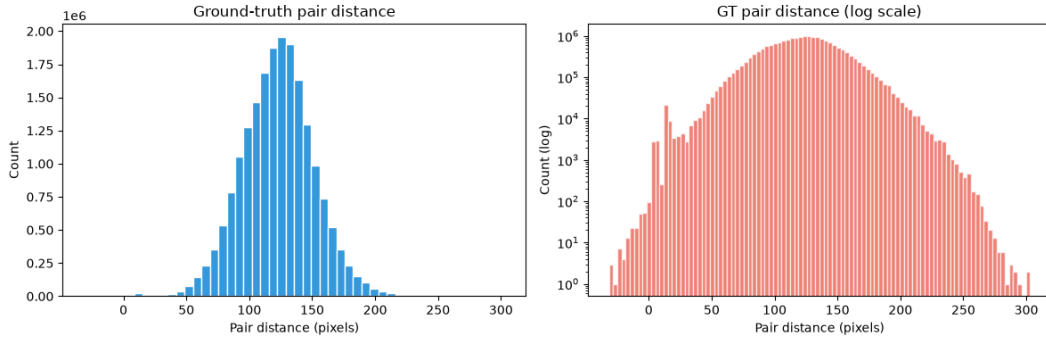


Figure 3: Distribution of pair-level ground-truth distances across the full corpus. The left panel uses a linear scale and shows a bell-shaped concentration around the mean spacing of approximately 115 px; the right panel uses a logarithmic scale to make the tails visible.

The First Pivot: From Sparse Kern Pairs to the Full N^2 Target

The first kerning experiments were run in a memory- and I/O-conscious regime. To keep training tractable, the pairwise model was limited to a small number of sampled pairs per font rather than all 2704. The practical motivation was real—long epochs, heavy I/O, and interrupted Colab sessions—but the consequence was a weak training signal and a visible generalization gap. In that setting the pairwise model reached a test MAE of about 5.54 pixels and the set-wise model about 7.85 pixels, which was neither competitive with the paper nor internally satisfying.

The fix was conceptual before it was computational. The reference paper targets *all* N^2 pairs, treating the absence of an explicit kerning entry as $k = 0$. That is also what real font rendering does. We therefore rewrote the rasterization target to iterate over every ordered pair and look up the kerning value, defaulting to zero when absent.

The immediate obstacle was scale. Materializing all 19.6 million pairs as a pandas DataFrame would have cost several gigabytes of memory. We avoided that with *lazy* N^2 indexing: the dataloader stores one cumulative offset per font, then maps a global pair index to a font and a local pair through a binary search plus modular arithmetic. The offsets for the training fonts occupy only a few tens of kilobytes, while any pair remains addressable in effectively constant time.

This change transformed the results. The pairwise model went from roughly 5.5 to 3.89 pixels, and the set-wise model from 7.85 to 4.67 pixels. The lesson, which reappears throughout the

project, is that for this task a dense, correctly specified target matters more than almost any architectural choice.

ResNet18 Pre-training and Feature Extraction

Following the paper, we encoded each glyph with a ResNet18 trained from scratch on 52-way glyph classification. The single-channel glyph image is repeated across three channels, and the final layer maps the representation to the 52 Latin classes. Training used cross-entropy, Adam with learning rate 10^{-3} , batch size 64, a maximum of 20 epochs, and an early-stopping patience of 5. It stopped at epoch 8, reaching 0.980 validation accuracy and 0.971 test accuracy.

After training, the classification head was removed and the frozen trunk became a feature extractor. Every glyph was mapped to a 512-dimensional vector and stored in NPZ files, with a manifest covering all 7263 valid fonts. These frozen features are shared by every downstream model, so their quality is a hard upper bound on everything that follows. The high classification accuracy indicates that the trunk produces reliable shape descriptors, and visual inspection confirmed that structurally similar letters land close in feature space—precisely the property a pairwise regressor needs.

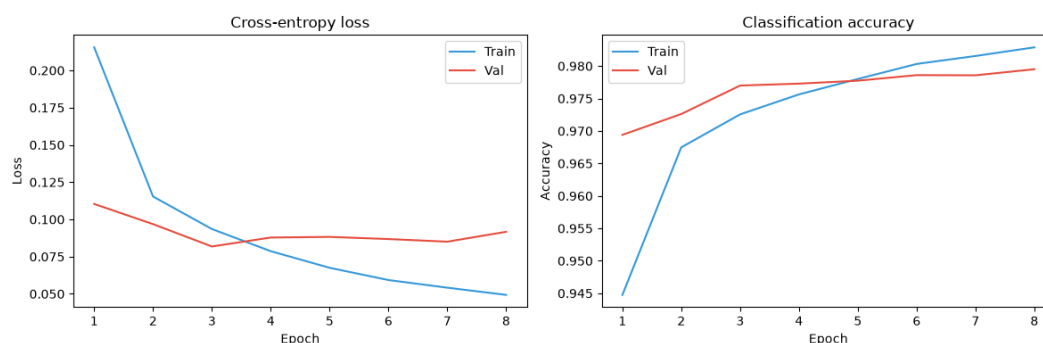


Figure 4: ResNet18 pre-training dynamics: cross-entropy loss on the left and classification accuracy on the right. Training stopped at epoch 8 under early stopping, with validation accuracy at 0.980.

Pairwise Model: The Strongest Single Architecture

The pairwise model was intended as the simple baseline; it turned out to be the strongest single architecture. Its input concatenates the left glyph’s 512-d feature, the right glyph’s 512-d feature, and two 52-d one-hot identity vectors, giving a 1128-dimensional conditional input. The predictor is a three-layer MLP with hidden dimension 256 and a scalar output, trained with MAE loss and Adam at learning rate 10^{-4} .

Trained on the full all-pairs dataset with lazy indexing, the model contains 355,073 parameters. It ran for 100 epochs with batch size 64 and an early-stopping patience of 20, reaching a best validation MAE of 3.8063 and a test MAE of 3.8949 pixels. This is a 70.7% improvement over the pairwise result in the reference paper (13.27 pixels) and, surprisingly, better than our own set-wise model.

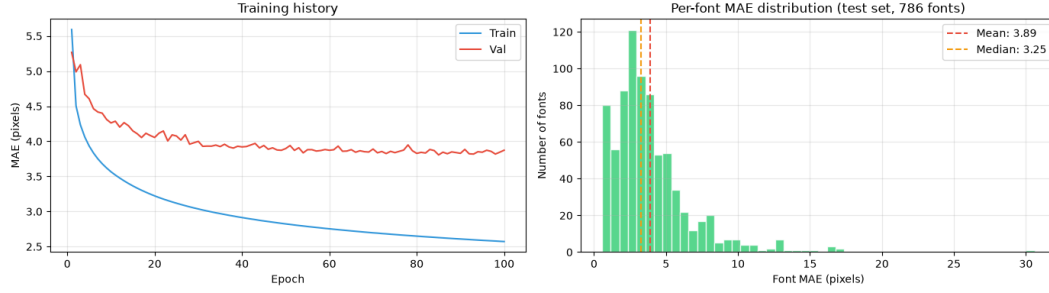


Figure 5: Pairwise training and validation MAE (left) and the per-font test MAE distribution over 786 fonts (right). The training and validation curves stay close and converge, without the large generalization gap seen in the earlier constrained run.

The main takeaway is that the pairwise formulation benefits enormously from dense supervision. Once the model sees every possible interaction among the 52 glyphs of each font, a plain DNN can learn both the average spacing of each class and the subtle shape-dependent corrections that kerning requires.

Set-wise Model: The Expected Advantage That Did Not Materialize

The paper’s headline result is that the set-wise model wins because self-attention sees the whole font. In our larger setup, that advantage disappeared.

The set-wise model operates on one font at a time. All 2704 ordered pairs are represented by concatenating the two 512-d glyph features, producing 2704 tokens of dimension 1024. A linear projection maps each token to 32 dimensions, and a compact Transformer encoder—one layer, two attention heads, feed-forward dimension 128, ReLU activation—contextualizes the sequence. A final linear head maps each embedding to a scalar spacing.

The model has 45,601 parameters. It was trained with Adam at learning rate 10^{-3} , batch size 64 fonts, and a maximum of 100 epochs, reaching a best validation MAE of 4.6803 and a test MAE of 4.6705.

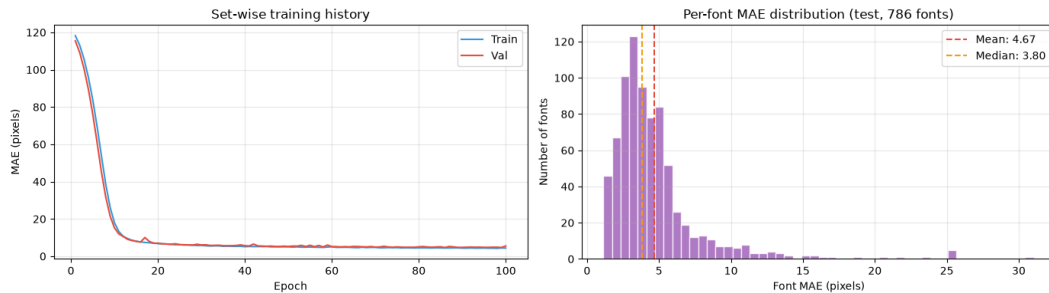


Figure 6: Set-wise training and validation MAE (left) and the per-font test MAE distribution (right). Both curves fall quickly and stay closely aligned, but the final error remains above the pairwise baseline.

In this all-pairs configuration the set-wise model is *less* accurate than the pairwise model. The self-attention maps are close to uniform: a single layer with $d_{model} = 32$ does not appear to have enough capacity to exploit global structure across 2704 tokens. This is the most important deviation from the reference results. It does not mean the set-wise idea is wrong; it means this

particular implementation is too small to recover the benefit the paper observed on a smaller dataset.

Ensemble and Hybrid Models: Searching for Global Signal

Once the pairwise model established itself, the natural question was whether the set-wise model still carried complementary information worth extracting. We evaluated three combination strategies.

The first is a plain average,

$$\hat{t}_{\text{ENS}}(L, R) = \frac{\hat{t}_{\text{PW}}(L, R) + \hat{t}_{\text{SW}}(L, R)}{2}, \quad (2)$$

which reaches a test MAE of 3.8804 pixels—marginally better than the pairwise alone. The improvement is concentrated on difficult fonts, where the pairwise geometry is least reliable.

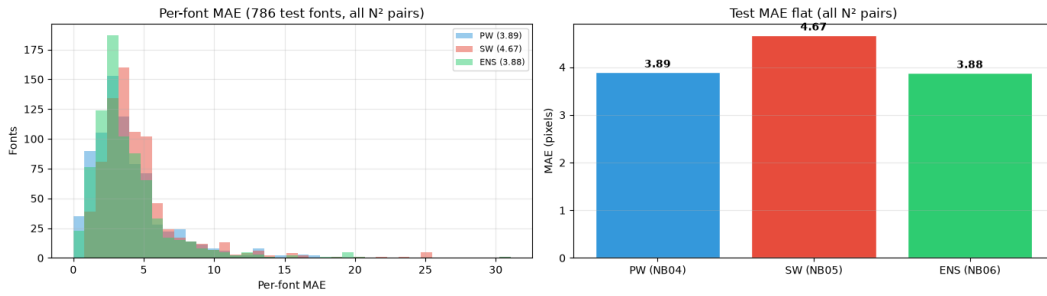


Figure 7: Per-font MAE distributions (left) and flat test MAE (right) for the pairwise, set-wise, and ensemble models. The ensemble distribution is the sharpest of the three, but its flat-MAE advantage over the pairwise model is only about 0.01 px.

The second strategy, set-to-pair fusion, appends a 32-dimensional global context vector—the mean of the frozen set-wise hidden states over all 2704 tokens—to the pairwise input, yielding a 1160-dimensional vector fed to the same three-layer MLP. It has 363,265 parameters and reaches a best validation MAE of 3.6226 and a test MAE of 4.0446. The context helps some hard fonts, but a single mean vector is too coarse to give a consistent advantage.

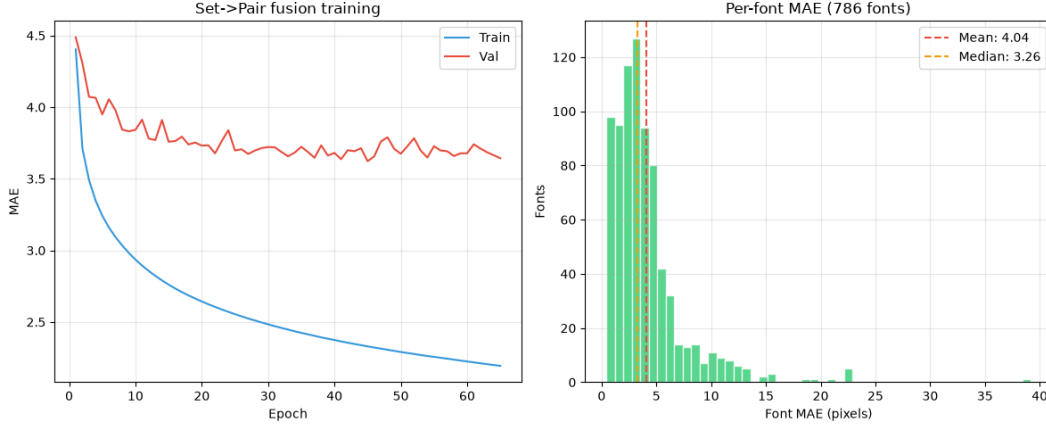


Figure 8: Set-to-pair fusion training dynamics (left) and per-font test MAE distribution (right). The validation curve fluctuates around 3.6–3.9 px after an early drop, and the test distribution is right-skewed with a long tail of hard fonts.

The third strategy, cross-attention, is the most expressive. A pairwise encoder maps the 1128-d input through $1128 \rightarrow 64 \rightarrow 64 \rightarrow 64$ into a query, which then attends over the frozen set-wise hidden states with learned Q , K , and V projections, and a small head predicts the scalar spacing. The model has 122,241 parameters and reaches a best validation MAE of 3.7039 and a test MAE of 4.1320. The attention distributions, however, are nearly uniform, indicating that the frozen hidden states do not provide a queryable representation for the pairwise problem.

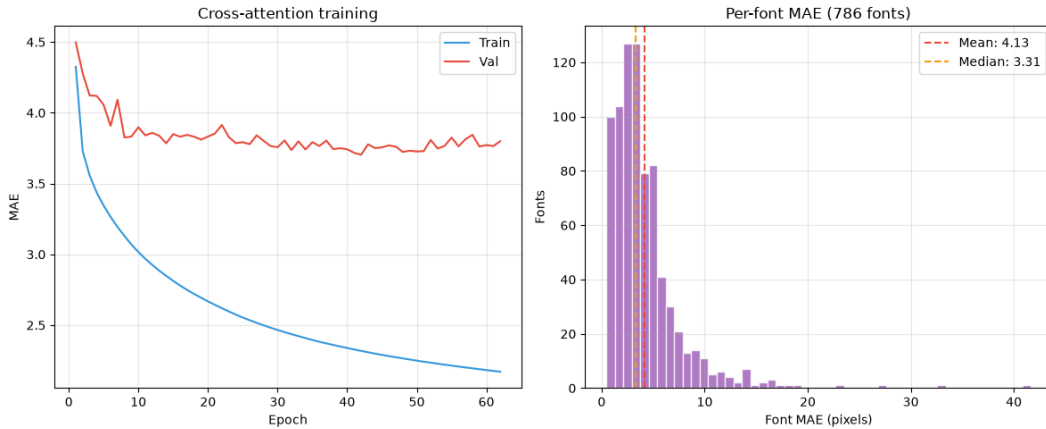


Figure 9: Cross-attention training dynamics (left) and per-font test MAE distribution (right). Despite a lower validation error, the test error remains above the pairwise baseline.

The comparison is instructive. The ensemble and the fusion model provide small, targeted gains on hard fonts, while cross-attention is a more complex way of reproducing the same averaging behavior. The frozen set-wise states are optimized to minimize their own scalar loss, not to serve as a transferable representation; that is the deeper reason the hybrids do not surpass the pairwise baseline.

Results

All models were evaluated on the same 786 test fonts and on all 2704 pairs per font, for 2,125,344 test predictions. The primary metric is the flat mean absolute error over all test pairs.

Model	Test MAE	Best val	Epochs	Parameters
Ensemble (PW+SW)/2	3.8804	—	—	—
Pairwise	3.8949	3.8063	100	355,073
Set-to-Pair Fusion	4.0446	3.6226	65	363,265
Cross-Attention	4.1320	3.7039	62	122,241
Set-wise	4.6705	4.6803	100	45,601

Table 1: Final kerning-estimation results on 786 test fonts and all 2704 pairs per font.

The pairwise model is the best single architecture and is effectively tied with the ensemble. Its test MAE of 3.89 pixels improves by about 70.7% over the paper’s pairwise result, while the set-wise model reaches 4.67 pixels, slightly better than the paper’s set-wise result of 5.32 pixels.

Per-style results use the categories returned by the Google Fonts API.

Style	n	PW	SW	ENS	SP	CA
Sans-serif	463	3.44	4.17	3.48	3.79	3.78
Serif	167	3.26	4.13	3.32	3.36	3.45
Display	80	6.83	7.13	6.35	6.43	6.81
Handwriting	76	4.93	6.30	4.95	4.60	4.94

Table 2: Mean absolute error by font style for the five evaluated models.

Sans-serif and Serif fonts are much easier than Display and Handwriting fonts. The ensemble helps most on Display, and the fusion model helps most on Handwriting. In head-to-head comparisons, the ensemble beats the pairwise on 406 of 786 fonts (51.7%), the fusion model on 407 (51.8%), and the set-wise model on 340 (43.3%). The global ranking still favors pairwise and ensemble because the hybrids’ gains are concentrated on hard fonts while their regressions elsewhere are larger.

The aggregate numbers hide where the models actually differ, so the following figures break the results down along three axes: letter pairs, font styles, and individual fonts.

The ground-truth spacing itself is highly structured. Figure 10 shows, for every ordered pair, the average target spacing and its variability across fonts. The mean map recovers the expected typographic pattern—wide pairs such as W–A and T–o, narrow pairs such as I–I and r–r—while the standard-deviation map identifies the pairs whose optimal spacing changes most from one font to another. These are precisely the pairs where a model cannot rely on a single average value and must condition on the glyph shapes.

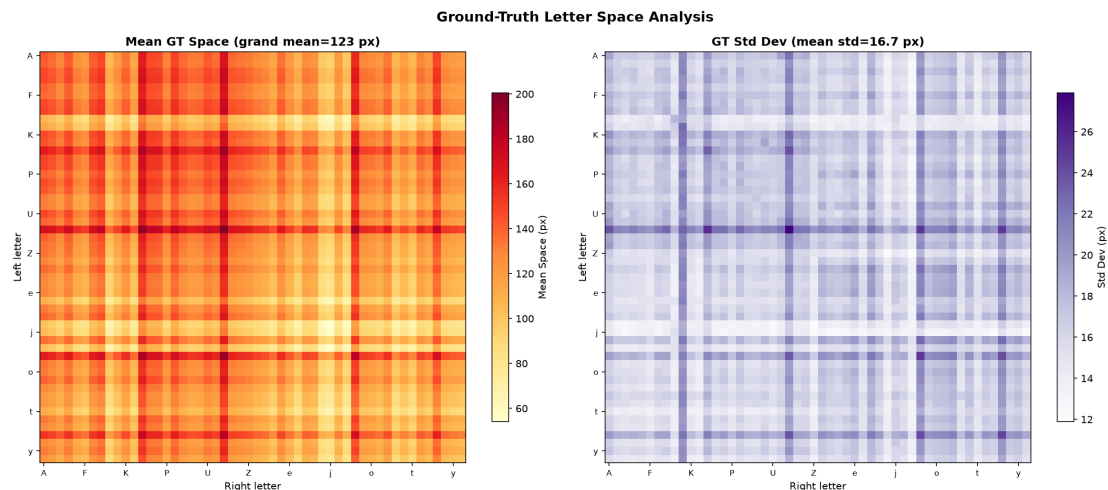


Figure 10: Mean (left) and standard deviation (right) of ground-truth letter spacing across all test fonts, computed over the 52×52 ordered pairs. The mean map captures the average spacing per pair class, while the standard-deviation map identifies which pairs vary most across fonts.

Figure 11 maps the per-pair MAE of each model onto the same 52×52 grid, with a final panel showing the pairwise-minus-set-wise difference. The five heatmaps share a common color scale, so the overall brightness of each panel is a direct proxy for that model's difficulty. The difference map makes the ordering concrete: most cells are blue, meaning the pairwise model is more accurate than the set-wise model across the large majority of letter pairs.

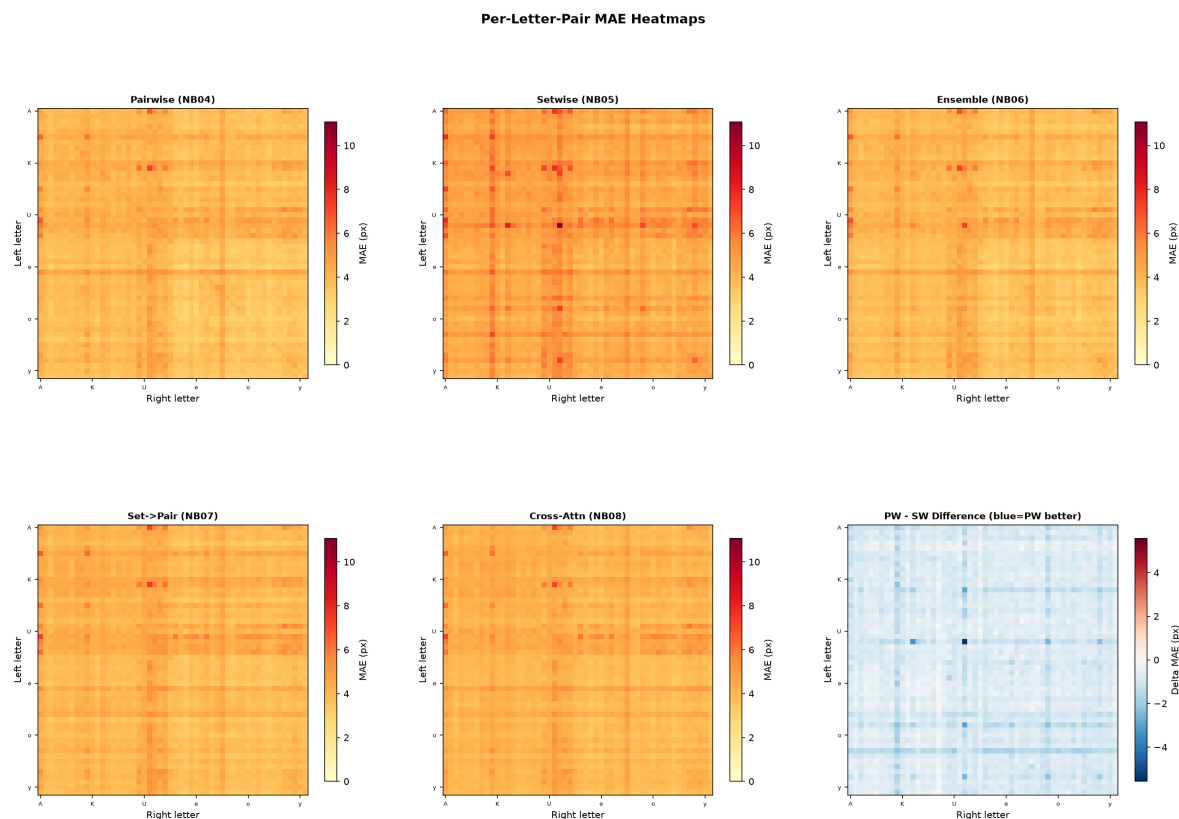


Figure 11: Per-letter-pair MAE heatmaps for the five models, plus the pairwise-minus-set-wise difference map (bottom right). Warmer cells indicate larger mean error; the difference map shows where the pairwise model wins (blue) versus the set-wise model (red).

A complementary view is the per-letter difficulty ranking in Figure 12. Averaging the error over all pairs that involve a given letter, the same characters keep reappearing at the hard end of the scale—W, V, T, and the lowercase **f**—because their diagonal or projecting strokes create the most variable spacing requirements. The five models agree on this ordering even where their absolute errors differ, confirming that the hard letters are a property of the task, not of a particular architecture.



Figure 12: Letter difficulty ranking across the five evaluated models. The upper panel ranks all models together; the lower panel isolates the pairwise and set-wise models. Hard letters are consistent across architectures.

The per-style breakdown in Figure 13 shows where the error is concentrated. Sans-serif and Serif fonts sit near 3.3–3.4 px for the pairwise model, while Display and Handwriting fonts reach roughly 6–7 px. The set-wise model is consistently worse in every style, and the ensemble reduces the error most on Display, the hardest category.

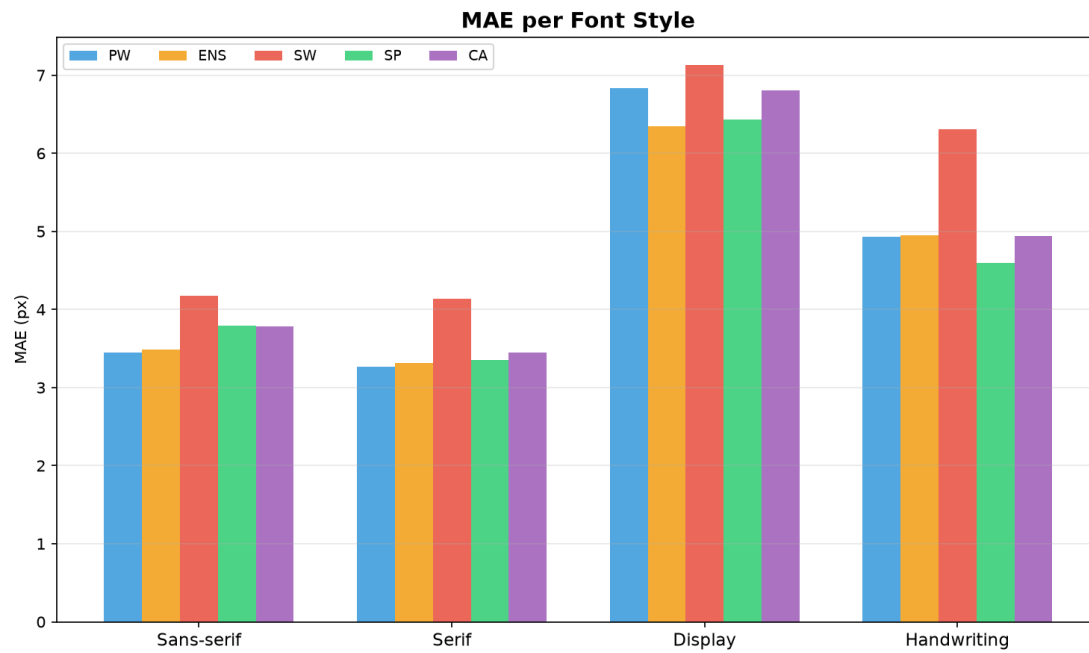


Figure 13: Per-style MAE comparison across all models. Sans-serif and Serif are consistently easier than Display and Handwriting; the ensemble lowers the error most on Display, while the fusion model does so on Handwriting.

Figure 14 shows the same ranking from a distributional perspective. The pairwise and ensemble curves dominate at every threshold: for example, the pairwise model keeps a larger fraction of predictions below any given pixel error than the set-wise model does. This confirms that the small mean difference is a consistent ordering across the whole corpus rather than an artifact of a few outliers.

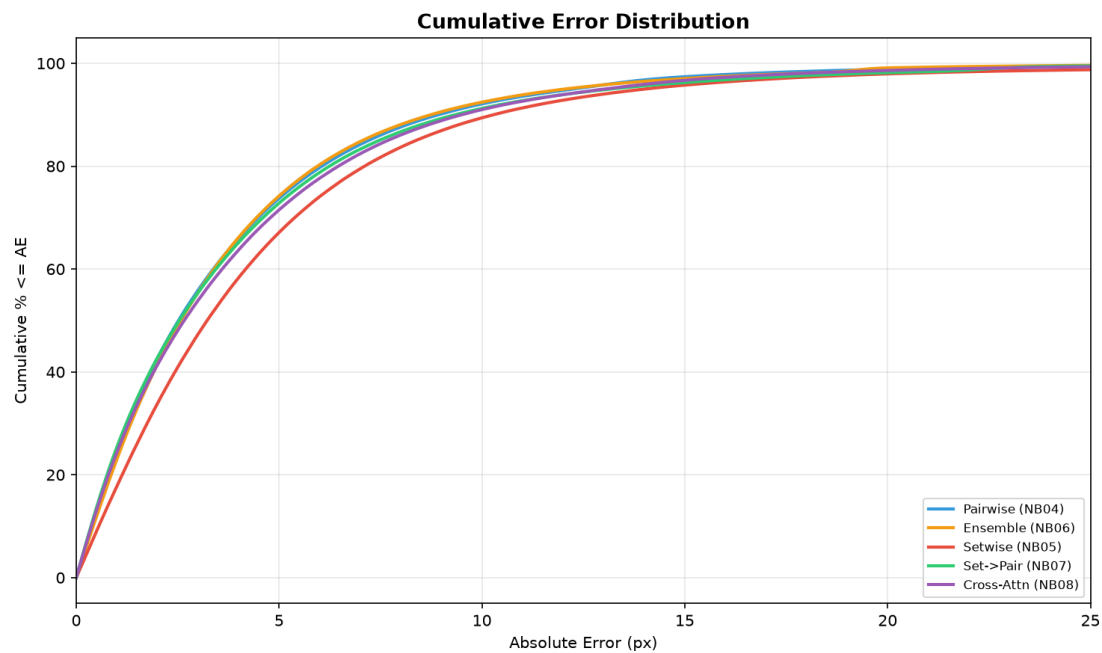


Figure 14: Cumulative error distributions for all five models. The pairwise and ensemble curves dominate at every threshold, confirming that the small mean difference reflects a consistent ordering rather than a few outliers.

At the font level, Figure 15 compares each hybrid and the ensemble against the pairwise model, one point per test font. Points below the diagonal are fonts where the alternative beats the pairwise baseline. The ensemble and the fusion model win on about half the fonts, but their victories are concentrated on high-error fonts, while the set-wise model loses on most fonts and only improves a minority.

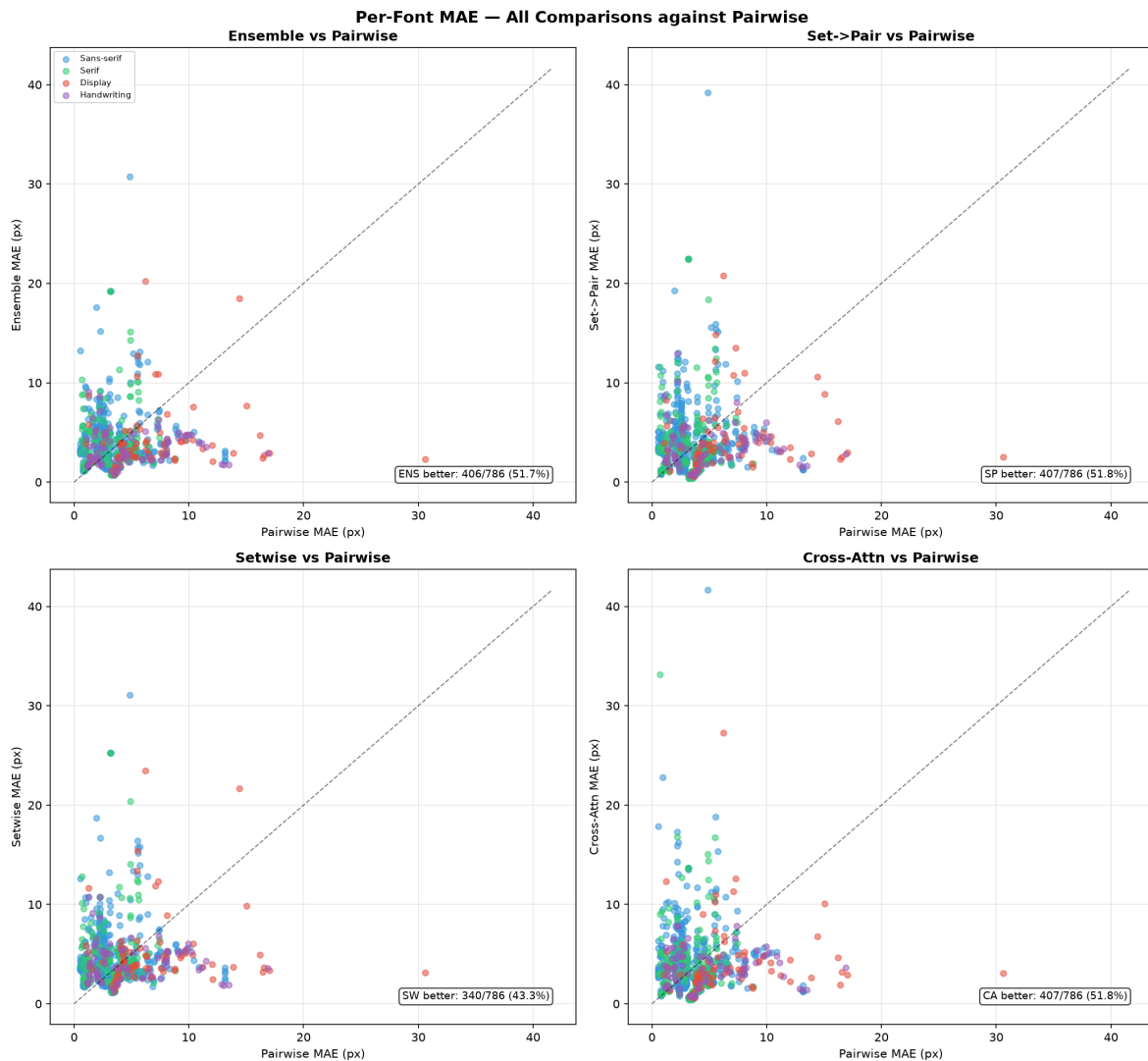


Figure 15: Per-font MAE of the hybrid models and ensemble compared against the pairwise model, colored by font style. Points below the diagonal indicate fonts where the alternative model is more accurate than the pairwise baseline.

Figure 16 zooms into the best and worst letter pair of each model. The easy pairs produce tight, low-error histograms centered near zero, while the hard pairs are wider and right-skewed, with a long tail of fonts where the model misses badly. The hard pairs are again dominated by characters with diagonal or projecting strokes, consistent with the letter ranking.

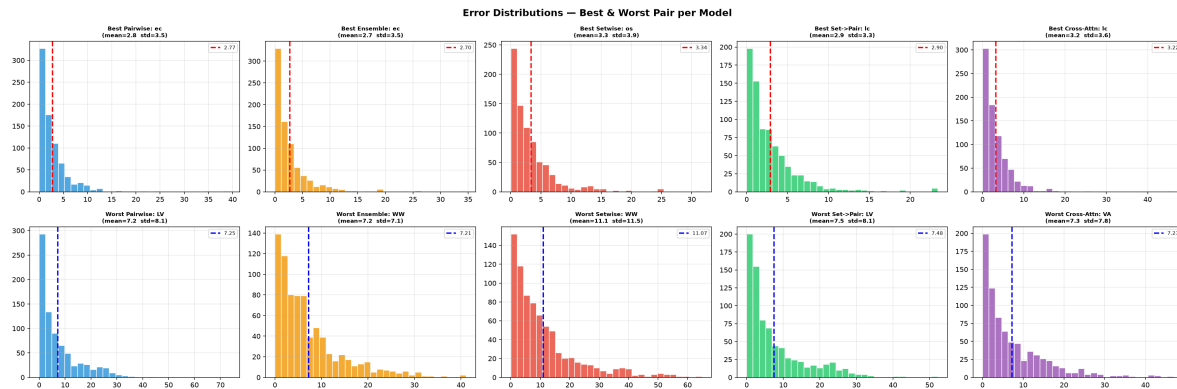


Figure 16: Error distributions for the best and worst letter pair of each model. Easy pairs (for example round-round combinations) produce tight distributions, while hard pairs (typically involving W, V, or T) are wider and right-skewed.

Finally, Figure 17 makes the residual error visible. For each style and model it overlays the ground-truth glyph placement in black with the model prediction in color, for both the best and the worst font. On the best fonts the two placements coincide almost perfectly; on the worst fonts the colored prediction drifts visibly away from the black ground truth, and the drift is largest for Display and Handwriting styles.

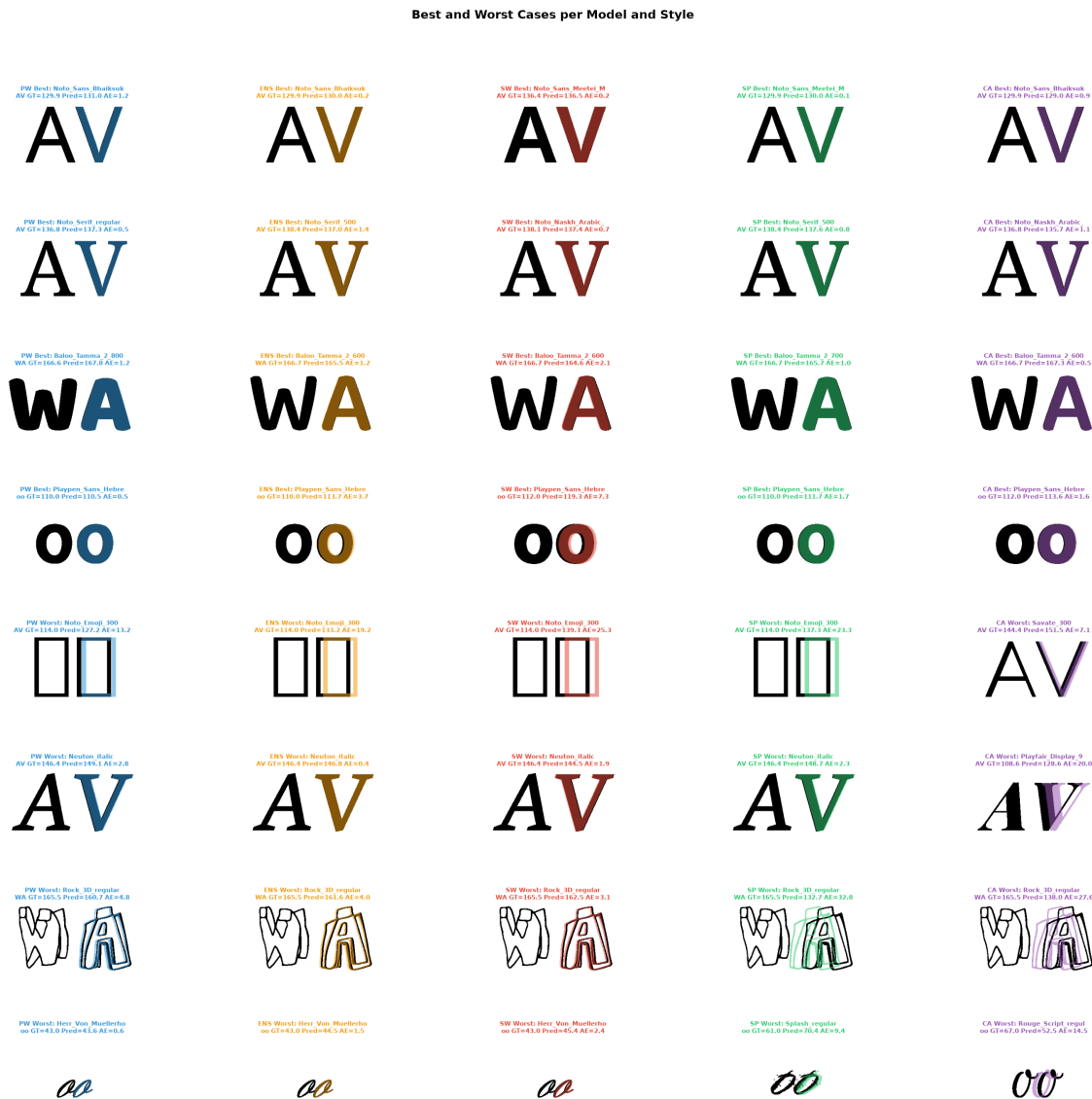


Figure 17: Best and worst cases per model and font style. Each cell overlays the ground-truth glyph placement (black) and the model prediction (colored), making the residual spacing error directly visible.

The overall interpretation is that the problem is dominated by the visual compatibility of the two glyphs. The pairwise model captures that local interaction well, while the global models add context that is not yet strong enough to improve the average error. The value of the ensemble is practical: it achieves the lowest error without any additional training, at the cost of running two models.

Explainability: What the Models Actually Learned

The explainability analysis makes the empirical story coherent: the pairwise model wins because it leans on shape, while the set-wise model stalls because it cannot form selective attention. The SHAP analysis of the pairwise model shows that the 1024 visual dimensions account for about 95.9% of total feature importance, while the 104 one-hot identity dimensions account for only

4.1%. The model relies on the shape of the two glyphs, not merely on which letters they are.

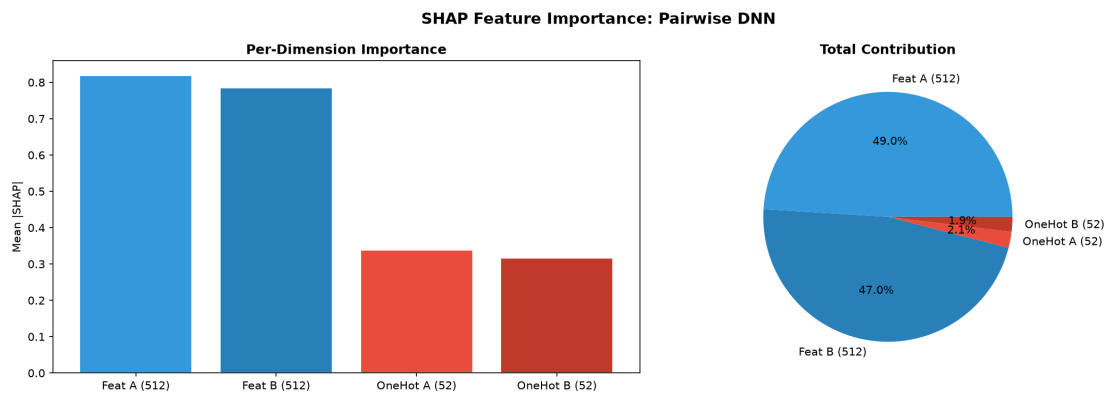


Figure 18: SHAP feature importance for the pairwise model. The left panel shows the mean absolute SHAP value per input group; the right panel shows the fraction of total contribution. Visual features dominate the one-hot identity features by a wide margin.

Grad-CAM on the ResNet18 backbone shows that the network attends to glyph edges and full silhouettes—exactly the regions that determine horizontal extent and therefore spacing.

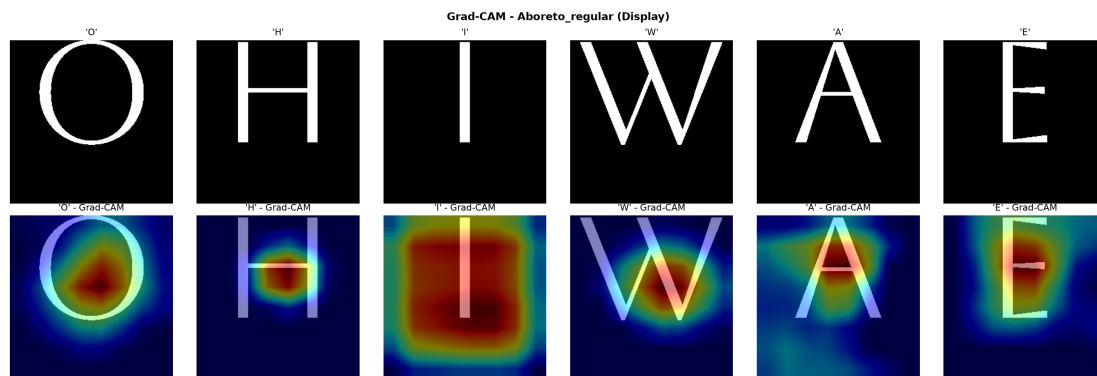


Figure 19: Grad-CAM visualizations for selected uppercase glyphs. The model concentrates on structurally diagnostic regions such as contours, crossbars, and stems, consistent with a shape-driven representation.

A t-SNE projection of the feature space provides a final qualitative check. Uppercase and lowercase glyphs form coherent structural clusters, with visually similar letters (such as O/Q or P/R) close together. This confirms that the frozen backbone encodes morphological similarity rather than a superficial per-font signal.

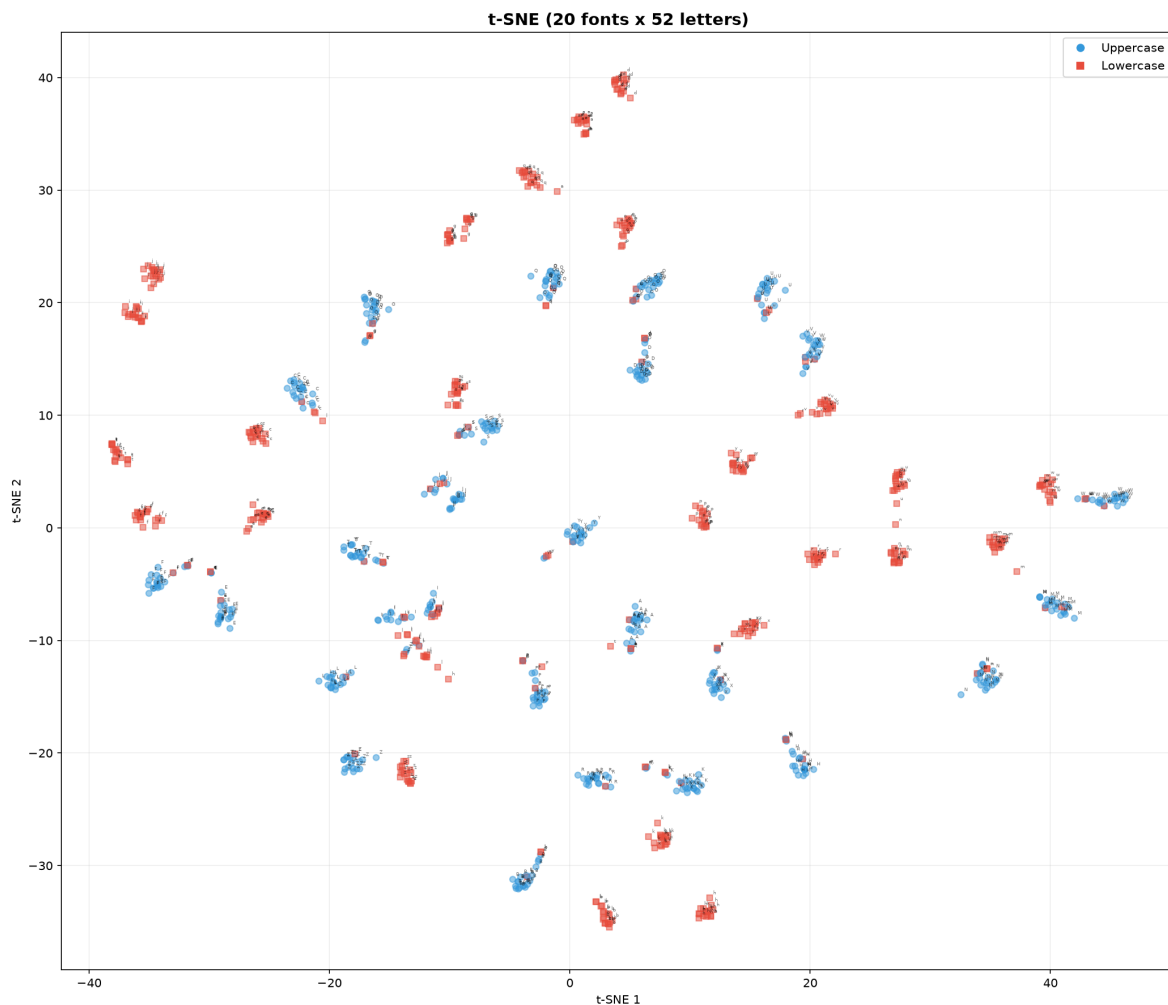


Figure 20: t-SNE projection of the ResNet18 feature space for 52 letters across 20 sampled fonts. Uppercase and lowercase glyphs separate into coherent clusters, and structurally similar letters land close together.

The self-attention maps of the set-wise Transformer are close to uniform for most fonts, with high entropy relative to the maximum achievable. This limited capacity explains why the set-wise model does not outperform the pairwise model here.

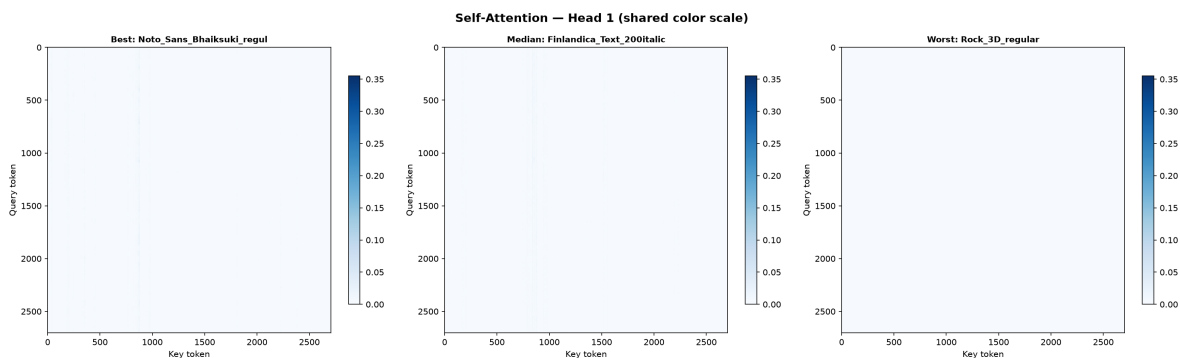


Figure 21: Self-attention maps of the set-wise Transformer for the best, median, and worst fonts by pairwise MAE. The maps are close to uniform, indicating that a single layer with $d_{model} = 32$ cannot form selective attention over 2704 tokens.

The cross-attention model also produces nearly uniform distributions over the frozen set-wise hidden states, and different pair queries do not lead to meaningfully different attention profiles. This supports the conclusion that the frozen set-wise representation does not contain easily queryable information for the pairwise regression task.

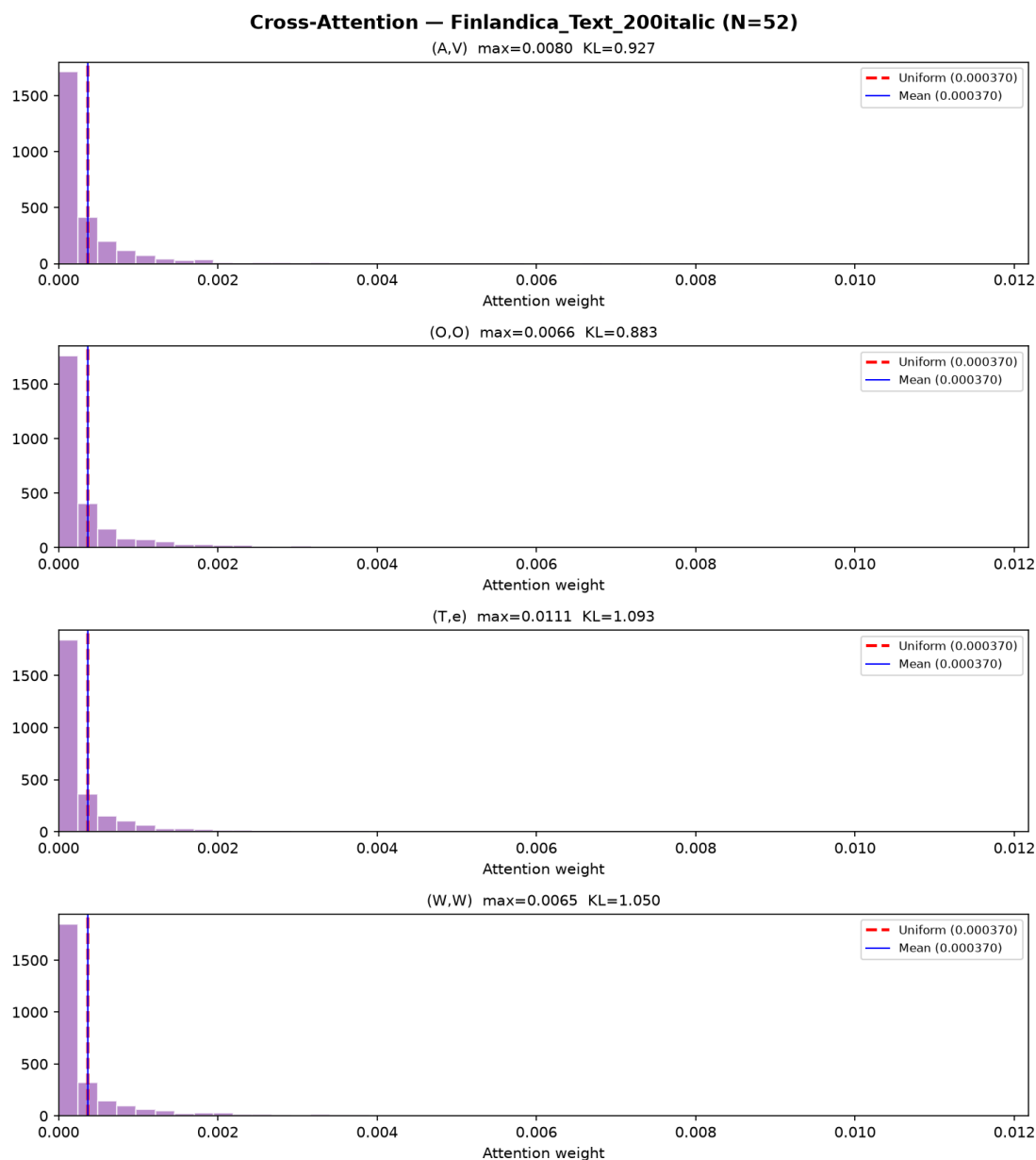


Figure 22: Cross-attention weight distributions for representative letter pairs. The distributions are indistinguishable from uniform (the red dashed line), with near-zero KL divergence, confirming that the frozen set-wise hidden states carry no queryable signal for the pairwise task.

Interactive Demo

The graphical interface, *DLKern*, is built with Tkinter and Matplotlib and offers two modes. In *Pair* mode the user selects a font and an ordered glyph pair, chooses which models to evaluate, and sees a per-model rendering followed by an overlay that places all predictions on the same pair. In *Word* mode the user enters a short text, and the demo computes spacing for every

adjacent pair, rendering the whole word with the ground truth and each selected model.

The demo uses lazy model loading and caches font features, glyph data, set-wise predictions, and hidden states, so switching models or fonts does not recompute the expensive set-wise forward pass. The available fonts are discovered from the processed feature files, and the inference functions are isolated in a separate module so the checkpoints can be reused outside the notebooks.



Figure 23: DLKern pair-mode screenshot for the pair **Gv**, showing the ground-truth rendering and the selected model overlays on the same glyph pair.

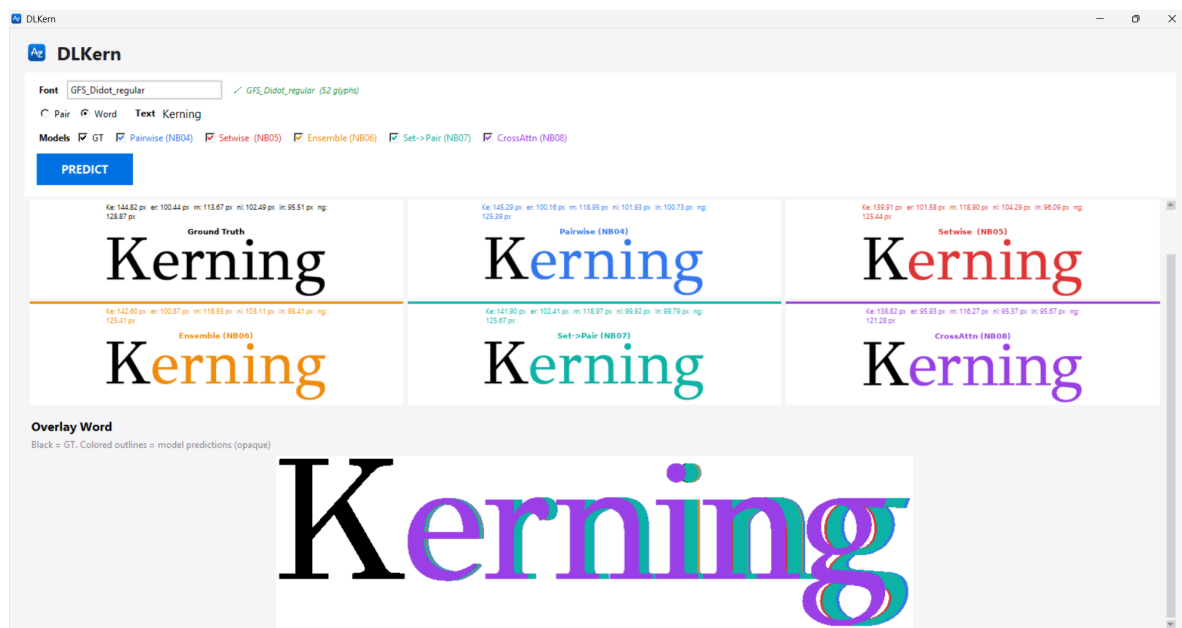


Figure 24: DLKern word-mode screenshot for the word **Kerning**, with each adjacent pair spaced by the ground truth and the selected models.

Conclusions

The project confirms that automatic kerning can be learned effectively from a large corpus of proportional Latin fonts. The pairwise conditional model is the strongest single architecture, reaching 3.89 pixels and improving by 70.7% over the paper’s pairwise result. A simple average ensemble with the set-wise model gives the best overall test MAE of 3.88 pixels, but the margin over the pairwise model is marginal and concentrated on difficult fonts. The set-wise model and the two hybrids do not outperform the pairwise model in this all-pairs configuration, despite their stronger global modeling assumptions.

Discussion

Three observations are worth carrying into future work. First, a correctly specified target beats a clever model: sparse kern-only supervision produced mediocre results, while the full N^2 target, with $k = 0$ for absent pairs, unlocked the pairwise model’s performance. Second, data scale matters more than architecture here: a three-layer DNN on 5,758 training fonts outperforms a Transformer, reversing the paper’s conclusion in our larger setting. Third, frozen hidden states are not automatically transferable: a representation optimized for one scalar loss is not a queryable feature space for another model, and both mean pooling and cross-attention failed to extract useful signal from it.

The practical output is a complete and reusable pipeline, from font download to trained checkpoints and interactive evaluation. The models are compact enough to deploy locally, and the demo shows that pairwise predictions are already visually close to ground truth on typical fonts. The remaining errors concentrate on decorative and handwritten styles, where even human-defined kerning is inherently more variable. A style-conditional ensemble remains the most natural next step: the current results already show that the ensemble and the fusion model are conditionally useful on Display and Handwriting fonts, so restricting their use to those styles could turn their local advantages into a reliable global gain.

On which figures are strictly necessary. The figures collected here document the full pipeline, and they are all useful to keep together in this working report. For the final document, however, only a subset is strictly necessary to support the argument, and the choice depends on what each figure contributes.

The data-acquisition figures—the kerning-value distribution, the rasterization sample, and the ground-truth distance distribution—describe the dataset rather than the results; they are worth keeping in a methods section, but the rasterization sample in particular can be dropped if space is tight, since a single-font glyph grid adds little once the rasterization procedure is described in words.

The training-history plots document convergence and can be omitted once the final metrics are reported, unless the goal is to show that the models were not overfitting.

The explainability figures split into two groups. The SHAP importance and Grad-CAM maps carry positive evidence about what the model relies on, and they support the claim that shape, not letter identity, drives the prediction; these are worth retaining. The t-SNE projection is a qualitative confirmation of the same point and is more dispensable.

The self-attention and cross-attention weight maps document a negative result: they show near-uniform attention with no selective structure, which is already stated quantitatively by the MAE table and the error heatmaps. They are the clearest candidates for omission because

they visualize an absence of pattern rather than a finding, and they add little to a reader who already accepts the numbers.

Finally, the results figures—ground-truth heatmaps, per-model error heatmaps, per-style MAE, cumulative error distributions, error distributions per letter pair, and best/worst cases per style—form the minimal visual evidence of the paper and should be retained. The letter-difficulty ranking and the per-font scatter plots are useful diagnostics, but they restate what the error heatmaps and the per-style table already show, so they can be cut if page constraints require it.

References